pgemu examples

This is a short guide to pgemu for someone using it for the first time. `README.md` next to it is the full reference, and `MEASUREMENTS.md` keeps the numbers people have measured with it. Read those when you need more than the basics.

Everything here is the same on any machine. The one exception is the last section, Mac tips, which is what one person found on a Mac in one sitting.

1. Introduction

pgemu runs a real calculator firmware image on your computer. It does not build the code for your machine the way the GTK simulator does. It takes the exact `.pgm` or `.pg5` file that goes on the calculator and runs it inside an emulated ARM chip. The DMCP system calls are answered in Python.

So pgemu shows you what the calculator itself does, not what a host build does. Use it when you need the real stack depth, the real instruction count, the exact pixels the screen receives, or the behaviour of the DMCP-only code. A run takes about a second, and for the standard calculators you need no ARM compiler.

2. Background

The GTK simulator builds the C47 source for your computer. That is quick, but it is not the calculator. Four things differ:

- GMP uses 64-bit limbs on a computer and 32-bit on the calculator. The arithmetic then takes different internal routes.
- The memory pool is 256 kB on a computer and 64 kB on the DM42.
- Every piece of code inside `#if defined(DMCP_BUILD)` is left out of the simulator.
- The simulator takes its screenshot from the lcd buffer, so a stray write that never reached a refresh still shows up in the picture.

pgemu has none of these, because it runs the real ARM binary. It does not replace the simulator. Use the simulator for quick work on maths and menus. Use pgemu for the things only the real hardware shows.

3. Requirements

- Python 3 and the `unicorn` package, 2.1 or newer. See `requirements.txt`.
- No ARM compiler to run a built image. The small ARM helper code is already built and checked in under `prebuilt/`. Without it, Python answers those calls, which is slower but gives the same result.
- An ARM compiler and the DMCP SDK only if you rebuild `prebuilt/` or run `--selftest`.
- A built image: `.pgm` for the DM42, `.pg5` for the R47. Build it with the project's make targets.
- Two files next to the image, with the same name: `<name>_qspi.bin` and `<name>.elf`. The DM42 needs the qspi file for its fonts and tables. The elf file is for `--profile` and for watching functions. Use `--qspi` or `--elf` if they sit somewhere else.
- A DMCP platform image, only if you want DMCP's own memory allocator and screen code to run. The R47 one is in `res/combo/`. The DM42 one comes from SwissMicros, at `https://technical.swissmicros.com/dmcp/firmware/`. Point to it with `--platform`.

4. Instructions to use

Build an image, then run pgemu on it with a list of keys to press:

```
python3 tools/pgemu/pgemu.py <image> --press '<keys>'
```

`<image>` is the image you built, for example `build.dmcp.p4/src/c47-dmcp/C47.pgm` for the DM42, or `build.dmcp5/src/c47-dmcp5/R47.pg5` for the R47.

`--press` reads the keys from the command line. `--script` reads them from a file.

The key names match the simulator's `press` command, written up in `res/SCRIPTS/cli_automation_examples.txt`. A script you wrote for the simulator runs here as it is. The key names are:

- a digit or letter presses that key; a lower-case letter means f-shift first;
- `ENTER`, `EXIT`, `R/S`, `ADD`, `SUB`, `MUL`, `DIV`, and the other named keys;
- `F1` to `F6` for the soft keys; `@f` and `@g` for the two shifts; `@k NN` for a key by its number;

- `wait N` idles for N rounds; `snap name` saves a screenshot; `mark name` resets the counters, so you then measure only what comes after it.

Separate the steps with `;` or a new line. A `#` starts a comment. Each key goes in with its release, one per round.

Put the keys on their own line, never in a sentence. For example, this script presses ENTER, types 3, f-shifts, presses soft key F1, waits, and saves a screenshot:

```
ENTER; 3; @f; F1; wait; snap example
```

To run a function or program by name, press XEQ twice, type the letters, then ENTER. The name SIN is three keys:

```
XEQ; XEQ; S; I; N; ENTER
```

To load a program file, put it in `tools/pgemu/fs/PROGRAMS/` and run READP the same way. When the file-selection screen appears, `--choose` answers it with the calculator's own backslash:

```
--choose 'PROGRAMS\NAME.p47' --press 'XEQ; XEQ; R; E; A; D; P; ENTER'
```

A run ends at the first of these, and the report names which one:

- the screen settles and the calculator sleeps with no key waiting;
- the program turns the calculator off;
- the firmware resets itself;
- it runs out of instructions (200 million by default; `--max-instructions 0` lifts the limit);
- it touches memory that is not mapped.

Give `--max-instructions 0` to anything that integrates, solves, plots or differentiates. Those go past the default limit.

Screenshots land in `tools/pgemu/out/`, or wherever `--out` points. Each run prints a report: which image ran, where it stopped, how much stack and heap it used, and every DMCP call it made.

Always read the `built from` line at the top of the report before you trust a run. It states which commit the image was built from. An old image left in a build folder can be weeks behind your branch.

To press the keys yourself, add `--serve` and open the web page it prints. You get the screen and a picture of the keyboard, with the stack and heap shown as bars down the side. Ctrl-c ends it.

5. Run a program in 3 minutes

The quickest way to see pgemu work is to run one of the programs in `res/PROGRAMS/`. These steps run SPIRALk, which draws a spiral.

Do the first two once:

1. Install `unicorn`. On macOS read the Mac tips first, then install it there.

```
pip install "unicorn>=2.1"
```

2. Build an image. Either hardware works:

```
make dmcp5r47     # the R47 image, build.dmcp5/src/c47-dmcp5/R47.pg5
make dmcp         # the DM42 image, build.dmcp.p4/src/c47-dmcp/C47.pgm
```

Then, each time:

3. Copy the program next to pgemu's files:

```
cp res/PROGRAMS/SPIRALk.p47 tools/pgemu/fs/PROGRAMS/
```

4. Run it and save a picture:

```
python3 tools/pgemu/pgemu.py <image> --max-instructions 0 \
    --choose 'PROGRAMS\SPIRALk.p47' \
    --press 'ENTER; wait 60;
             XEQ; XEQ; R; E; A; D; P; ENTER; wait 120;
             XEQ; XEQ; S; P; I; R; A; L; @f; k; ENTER;
             wait 600; snap spiral'
```

The letters after each `XEQ; XEQ;` spell a name. `R E A D P` runs READP, which loads the file named by `--choose`. `S P I R A L` then `@f; k` runs the label SPIRALk; the final `k` is lower case, so `@f` gives the f-shift and `k` presses the key.

To press the keys yourself instead, open the browser page:

```
python3 tools/pgemu/pgemu.py <image> --choose 'PROGRAMS\SPIRALk.p47' --serve
```

Open the web address it prints, click the page once, then on your own keyboard:

1. Press XEQ, type `READP`, press Enter. SPIRALk loads.
2. Press XEQ, type `SPIRAL`, press the `k` key, press Enter. The spiral draws.
3. Press any key to end it.

6. Benefits

- **Stack depth on the real chip.** `--stack-watermark` fills the unused stack with a pattern and reports how far down the run wiped it. Put a `mark` first, and the figure is for that routine alone.
- **Instruction counts.** `--profile` counts every instruction and groups them by function, so you see where an operation spends its time.
- **What the screen receives.** pgemu tracks what each refresh sends to the panel, and shows where a hand-off to one of DMCP's own screens left part of the display behind.
- **The DMCP-only code.** Everything behind `#if defined(DMCP_BUILD)` runs here and nowhere else off the calculator.
- **The real allocator.** With a platform image, memory comes from DMCP's own allocator. On the DM42, a stack that runs too deep writes into the heap and breaks it, the same failure as on the calculator. The report names the first bad write.
- **RAM-full paths without filling RAM.** `--malloc-fail`, `--malloc-fail-above` and `--fail` refuse a chosen allocation or function, so you reach the out-of-memory path on purpose.

A run costs about a second, so you can check one build per commit across a long history, with no flashing.

7. Examples

Each block is one command. The line under it states what you get.

Add 3 and 4 on an R47 image:

```
python3 tools/pgemu/pgemu.py build.dmcp5/src/c47-dmcp5/R47.pg5 \
    --press 'ENTER; 3; ENTER; 4; ADD; wait; snap add'
```

Saves `out/add.bmp` with 7 in X.

Press the keys yourself in a browser:

```
python3 tools/pgemu/pgemu.py build.dmcp5/src/c47-dmcp5/R47.pg5 --serve
```

Profile opening a menu:

```
python3 tools/pgemu/pgemu.py <image> --profile \
    --press 'ENTER; wait; mark menu; <keys to open it>; wait'
```

The report states how many instructions the menu took, and which functions used them.

Measure how deep a program's stack goes:

```
python3 tools/pgemu/pgemu.py <image> --max-instructions 0 \
    --stack-watermark --choose 'PROGRAMS\MINE.p47' \
    --press 'ENTER; wait 60;
             XEQ; XEQ; R; E; A; D; P; ENTER; wait 200;
             mark run;
             XEQ; XEQ; M; I; N; E; ENTER; wait 4000'
```

The `mark run` keeps the depth to the program itself, not the loading before it.

Compare two builds, frame by frame:

```
python3 tools/pgemu/pgemu.py <old> --ms-per-tick 1 \
    --clock '2026-01-01 12:00' --frames old.txt --press "$KEYS"
```

```
python3 tools/pgemu/pgemu.py <new> --ms-per-tick 1 \
    --clock '2026-01-01 12:00' --frames new.txt --press "$KEYS"
cmp old.txt new.txt
```

Fixing the clock makes both runs identical, so the first line `cmp` reports is the first frame that changed.

Watch one firmware function:

```
python3 tools/pgemu/pgemu.py <image> \
    --entry-sp <function> --calls <function> \
    --press '...; mark m; ...'
```

Reports how deep that function was entered, and how often, since the mark.

Force an out-of-memory failure on a DM42 image:

```
python3 tools/pgemu/pgemu.py <image>.pgm --platform <DM42 DMCP>.bin \
    --max-instructions 0 --malloc-fail-above 8192 \
    --press 'ENTER; wait 60; ...load and run...; wait'
```

Refuses any block over 8192 bytes, and names the code that asked for it.

8. Tips

- A program that keeps testing `KEY?` with no pause never lets a scripted key in. It stays busy, so it never sleeps, and a key goes in only while the firmware waits. The run then hits the instruction limit. Press the key during a `PAUSE` instead, where the program does wait.
- `PAUSE 99` waits for a key for ever. A scripted run that reaches it never settles on its own. Use a short `PAUSE`, or let the program finish.
- To capture a program's own graphics, take the shot while the program is still running. Once it returns, the stack is painted back over the screen. A `SNAP` step inside the program, or the frame list, catches the graphics in time.
- `--dmcp` runs DMCP's own screen code from the platform image. For C47's own screens the result is the same either way, so you need it only when the screen is one of DMCP's own, or when you want the instruction count to match the calculator.
- Item numbers in a hand-written `.p47` change from build to build. Read them from the build you are running, or call the function by name.
- When you compare two builds, compare the byte counts on their `prog_info` lines, not the file sizes. A `.pgm` on disk is 20 bytes longer than that count.
- A tree checked out with `git archive` has no `.git`. The `built from` line then shows the commit of whatever repository sits around it, not the one you built. Tell such images apart by their sha256.
- A build made with `STACK_WATERMARK` writes its own pattern over pgemu's. `--stack-watermark` then reports that, not a real depth. The report warns you when it happens.
- A missing-refresh bug shows up on one route only. The same answer reached two ways can leave the status bar blank one way and not the other.
- A DM42 run with no platform image uses your computer's allocator. A stack overrun into the heap then does not crash, where on the calculator it would.
- On the DM42 the engines nest only two deep. PLOT over INT over INT gives "Operation aborted". That is the firmware, not pgemu.
- On the C47 keyboard, f then EXIT is OFF. A script that presses it ends the run early, before any later `snap`.
- A run is quick enough for `git bisect run`: build one image per step, then let a `grep` of the report pass or fail that commit. Return exit 125 on a build that fails, so bisect skips that commit rather than counts it against the fault. Keep a copy of pgemu, and the DMCP platform image, outside the checkout, so both are there on commits older than the day pgemu was merged.

9. Mac tips

These are from one session on a Mac, inside an agent's command sandbox. They are the Mac-only snags, with the exact commands another agent on this machine needs. `<scratch>` below is any folder you can write to, for example a temporary working folder, and `<venv>` is the virtual environment you make inside it. Write both out in full when you use them.

Unicorn is killed inside the sandbox, with exit code 132. pip also fails the TLS certificate check inside it, with error OSStatus -26276. So make the virtual environment and run every pip and pgemu command outside the sandbox.

Make a throwaway virtual environment and install unicorn into it:

```
python3 -m venv <scratch>/venv
<scratch>/venv/bin/pip install --no-cache-dir "unicorn>=2.1"
```

Run pgemu with that environment's python, not the system one. The Mac's default `python3` here did not have unicorn:

```
<scratch>/venv/bin/python3 tools/pgemu/pgemu.py <image> --press '...'
```

If the command gate keeps asking before each run, add two allow rules to `.claude/settings.local.json`, with the venv path written out in full: one for `<venv>/bin/pip install *` and one for `<venv>/bin/python3 .../pgemu.py *`.

Build the DM42 image with the Homebrew ARM compiler:

```
make dmcp
```

It writes `build.dmcp.p4/src/c47-dmcp/C47.pgm`, with `C47_qspi.bin` and `C47.elf` next to it. The first build of that folder compiles GMP and takes a few minutes; later builds reuse it.

A DM42 run with `--platform` and a DMCP image is much slower than one on the stubs, because DMCP's own screen code then runs in the emulator too. For a plain screenshot of C47's own screens the stubs are enough, and look identical.

10. Advanced examples

These examples keep a key down long enough for its longpress, and compare the capture the firmware writes with one pgemu takes.

A longpress with `hold`

`hold KEY MS` presses a key, keeps it down for MS milliseconds of the firmware's own clock, then releases it. A plain press releases the key in the same round, so a longpress never starts. A third word is a name for a capture, taken just before release. The capture shows which stage the key is on.

```
python3 tools/pgemu/pgemu.py build.dmcp.p4/src/c47-dmcp/C47.pgm \
    --press '5; ENTER; 7; hold @k 16 1000 stage'
```

With 7 still in number entry, `stage.bmp` shows CLN on the T line, the first stage of BACKSPACE, and the release clears the entry. Kept down for 200 ms the line shows BKSPC and the release deletes the one digit; kept down for 3200 ms it shows NOP and the release does nothing.

By default pgemu does not wait out the firmware's sleeps. Instead the sleep time is added to the clock and execution continues directly. This causes the `--press` script above to finish faster than the hold for 1000 ms. With `--ms-per-tick 1` the clock moves 1 ms each time the firmware calls `sys_current_ms` for the time. The captures above are the same either way.

The firmware's own capture file

SNAP calls DMCP's `create_screenshot`, which pgemu answers by writing the sent image into the mapped file system as `tools/pgemu/fs/DATA/<date>-<time>00.bmp`, named from the host's clock. SNAP writes the stack registers beside it, in a `.REGS.TSV` of the same name. The two keyboards reach SNAP by different keys.

On the C47, SNAP is g EXIT. The first stage of a longpress of EXIT there is MyMenu, not SNAP.

```
python3 tools/pgemu/pgemu.py build.dmcp.p4/src/c47-dmcp/C47.pgm \
    --press '355; ENTER; 113; DIV; @g; EXIT; wait; snap after'
```

On the R47, SNAP is the first stage of a longpress of EXIT. `stage.bmp` shows SNAP on the T line while EXIT is down, and the release runs it.

```
python3 tools/pgemu/pgemu.py build.dmcp5/src/c47-dmcp5/R47.pg5 \
    --press '355; ENTER; 113; DIV; hold EXIT 1000 stage; wait; snap after'
```

In both runs the firmware's capture is identical to the `snap` taken after it:

```
cmp tools/pgemu/fs/DATA/<date>-<time>00.bmp tools/pgemu/out/after.bmp
```

On a SHOW page EXIT acts on its release, on both keyboards. A short press leaves the page, and a longpress has SNAP at its first stage and NOP at its second, with the page drawn again after either.

```
python3 tools/pgemu/pgemu.py build.dmcp5/src/c47-dmcp5/R47.pg5 \
    --press '355; ENTER; 113; DIV; @f; DOT; wait; snap showpage;
             hold EXIT 1000 stage; wait; snap after'
```

`stage.bmp` shows SNAP over the first line of the page while EXIT is down. The firmware's capture, `showpage.bmp` and `after.bmp` are the same without the function name overlay.